# USEI12 - Análise de Complexidade Temporal

## Contexto

Para estimar uma base de custos para manutenção ou expansão, é necessário um subconjunto mínimo de trilhos que conecte todas as estações alcançáveis.

### User Story

> **Como planeador de infraestrutura**, quero calcular uma rede mínima sobre a rede ferroviária (tratada como não direcionada) para que eu possa identificar uma espinha dorsal mínima que conecte todas as estações com o comprimento de trilhos total mínimo.

---

## Análise da Complexidade Temporal

### Variáveis de Análise

- **n**: Número de vértices (estações)
- **m**: Número de arestas (linhas/conexões)

## Método Principal

---

### Graph<Station, Weight> minimalBackbone(Graph<Station, Weight> graph)

| Linha | Código | Complexidade Temporal | Justificação |
|---|---|---|---|
| 1 | **if(!graph.isConnected()){** | **O(n+m)** | Verificação da Conectividade através do algoritmo de DepthFirstSearch |
| 2 | **return null; }** | **O(1)** | Retorna se o grafo não for conexo |
| 3 | **return getMinimalSpanningTree(graph);** | **O($m^2$·n)** | Algoritmo de Kruskal para obter o grafo |

**Nota:** A complexidade temporal do método **isconected** e **getMinimalSpanningTree** são exploradas nas tabelas de métodos auxiliares abaixo.

**Complexidade Temporal Total:**

.: $O(n + m + 1 + m^2 \cdot n) = \mathbf{O(m^2 \cdot n)}$

# Métodos Auxiliares

---

## boolean isConnected( )

| Linha | Código | Complexidade Temporal | Justificação |
|---|---|---|---|
| 1 | **LinkedList dfs = Algorithms.DepthFirstSearch(this, vertices.get(0));** | **O(n+m)** | Verificação da Conectividade através do algoritmo de DepthFirstSearch |
| 2 | **return (dfs.size( ) == numVerts);** | **O(1)** | Comparação de inteiros, retorna verdadeiro se forem iguais |

**Nota:** A complexidade temporal do método **DepthFirstSearch** é explorada nas tabelas abaixo.

**Complexidade Temporal Total:**

.: $O(n + m + 1) = \mathbf{O(n + m)}$

---

## static <V, E> LinkedList DepthFirstSearch(Graph<V, E> g, V vert)

| Linha | Código | Complexidade Temporal | Justificação |
|---|---|---|---|
| 1 | **boolean[ ] visited = new boolean[ g.numVertices( )];** | **O(n)** | Criação e inicialização do array de vertices visitados |
| 2 | **LinkedList qdfs = new LinkedList<>( );** | **O(1)** | Criação de uma LinkedList vazia |
| 3 | **DepthFirstSearch(g, vert, visited, qdfs);** | **O(n+m)** | Chama o método que vai realizar o algoritmo |
| 4 | **return qdfs;** | **O(1)** | Retorna a lista quando esta está completa |

**Nota:** A complexidade temporal do método **DepthFirstSearch** é explorada na tabela abaixo.

**Complexidade Temporal Total:**

.: O(n + 1 + n + m + 1) = **O(n + m)**

---

## static <V, E> void DepthFirstSearch(Graph<V, E> g, V vOrig, boolean[ ] visited, LinkedList qdfs)

| Linha | Código | Complexidade Temporal | Justificação |
|---|---|---|---|
| 1 | **if(visited[g.key(vOrig)]) {** | **O(1)** | Acede ao Array e verifica se aquele vértice já foi visitado |
| 2 | **return; }** | **O(1)** | Retorna se o vértice já foi visitado |
| 3 | **qdfs.add(vOrig);** | **O(1)** | Adiciona um elemento à LinkedList |
| 4 | **visited[g.key(vOrig)] = true;** | **O(1)** | Marca o vértice com visitado |
| 5 | **for (V adj: g.adjVertices(vOrig)) {** | **O(grau(vOrig))** | Itera sobre todos os vértices adjacentes |
| 6 | **DepthFirstSearch(g, adj, visited, qdfs); }** | **O(1)** | Chamada recursiva para cada vértice adjacente não visitado |

**Complexidade Temporal Total:**

.: Cada vértice é visitado uma vez O(n) e cada aresta é explorada uma vez O(m) = **O(n + m)**

---

## Graph<Station, Weight> getMinimalSpanningTree(Graph<Station, Weight> graph)

| Linha | Código | Complexidade Temporal | Justificação |
|---|---|---|---|
| 1 | **Graph<Station, Weight> mst = new MapGraph<>(false);** | **O(1)** | Criação de um grafo vazio |
| 2 | **for (Station station : graph.vertices()) {** | **O(n)** | Itera sobre todos os vértices do grafo |
| 3 | **mst.addVertex(station); }** | **O(1)** | Adiciona o vértice ao grafo |
| 4 | **Collection<Edge<Station, Weight>> edges = graph.edges();** | **O(m)** | Obtém todas as arestas do grafo |

| Linha | Código | Complexidade Temporal | Justificação |
|---|---|---|---|
| 5 | **List<Edge<Station, Weight>> edgeList = new ArrayList<>(edges);** | **O(m)** | Cria uma lista com todas as arestas |
| 6 | **edgeList.sort(Comparator.comparing(Edge::getWeight, Comparator.comparingDouble(Weight::getDistance)));** | **O(m log m)** | Ordena os elementos da lista de arestas por peso |
| 7 | **for (Edge<Station, Weight> edge : edgeList) {** | **O(m)** | Itera sobre todas as arestas ordenadas |
| 8 | **LinkedList connectedVertices = Algorithms.DepthFirstSearch(mst, edge.getVOrig());** | **O(n+m)** | Executa o algoritmo de DepthFirstSearch para verificar a conectividade |
| 9 | **if (!connectedVertices.contains(edge.getVDest())) {** | **O(n)** | Verifica se o vértice de destino está na lista |
| 10 | **mst.addEdge(edge.getVOrig(), edge.getVDest(), edge.getWeight());** | **O(1)** | Adiciona a aresta ao grafo |
| 11 | **return mst;** | **O(1)** | Retorna o grafo |

**Complexidade Temporal Total:**

.: $O(1 + n + m + m + m \log m + m \cdot (n+m+n+1)) = O(m \log m + m \cdot (n+m+n)) = O(m \log m + m^2 \cdot n) =$ **$O(m^2 \cdot n)$**

# Considerações Finais

A análise de complexidade temporal do método **minimalBackbone** revela que a operação mais custosa é a obtenção da árvore geradora mínima utilizando o algoritmo de Kruskal, resultando em uma complexidade total de **$O(m^2 \cdot n)$**. Este valor é crucial para entender o desempenho do sistema ao lidar com grandes redes ferroviárias, onde o número de estações e conexões pode ser significativo. A eficiência do algoritmo impacta diretamente na capacidade de planeamento e otimização da infraestrutura ferroviária.